Ders 23 Genişletilmiş Notu: Minimum Kapsayan Ağaç: Kruskal ve Prim, Dijkstra ile En Kısa Yol

Ders 23 Genişletilmiş Notu: Minimum Kapsayan Ağaç: Kruskal ve Prim, Dijkstra ile En Kısa Yol

Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Hatırlatma: Köy, Muhtar ve En Küçük Kapsayan Ağaç
- 2. Jenerik MST Algoritması ve Güvenli Kenar
- 3. Kruskal Algoritması
- 4. Kruskal ile Kitaptaki 9 Düğümlü Örnek
- 5. Kruskal'ın Karmaşıklığı
- 6. Prim Algoritması: Tek Ağacı Büyütmek
- 7. Prim ile Aynı Köy Örneği
- 8. Prim'in Karmaşıklığı ve Kruskal ile Karşılaştırma
- 9. Tek Kaynaklı En Kısa Yol Problemi
- 10. Dijkstra Algoritması
- 11. Dijkstra Örnek 1: Elle Çizilmiş Yönlü Graf
- 12. Dijkstra Örnek 2: Kitaptaki Yönlü Graf
- 13. Dijkstra Örnek 3: Yönsüz Graf ve A'dan Z'ye Yol

1. Hatırlatma: Köy, Muhtar ve En Küçük Kapsayan Ağaç

Hoca derse, graflar konusunun son başlığı olan en küçük kapsayan ağaç (minimum spanning tree, MST) konusunu hatırlatarak başladı. Türkçede "en küçük yayılım ağacı" ya da "en küçük örten ağaç" da denir.

Anlamayı kolaylaştıran örnek şuydu: Bir köyün muhtarıyız. Sorumluluğumuz köydeki bütün evleri birbirine bağlamaktır. Her ev bir düğümdür, iki ev arasında yapılabilecek her yol bir kenardır. Her yolun bir yapım (ya da bakım) masrafı vardır. Buna kenarın ağırlığı deriz ve $w(u, v)$ diye yazarız. Amacımız, bütün evleri birbirine bağlı tutan ve toplam masrafı en az olan bir yol ağı kurmaktır.

Köy yönsüz (undirected) bir graf olarak modellenir. Hoca, yönlü graf için de MST kurulabileceğini, ama o zaman A'dan B'ye erişim ile B'den A'ya erişimi ayrı ayrı düşünmek gerekebileceğini söyledi.

Ağırlık fonksiyonu $w : E \to \mathbb{R}$ biçimindedir, yani her kenara bir gerçek sayı atanır. Aradığımız şey, kenarların öyle bir alt kümesidir ki:

- Bütün düğümler birbirine bağlı kalır.
- Yapı bir ağaçtır, yani hiçbir döngü (cycle) içermez.
- Seçilen kenarların ağırlıkları toplamı olabilecek en küçük değerdir.

Ağaç olduğu için V düğümlü bir MST'de tam $V - 1$ kenar bulunur. Bu, bütün düğümleri bağlamak için gereken en az kenar sayısıdır. Döngü oluşursa iki düğüm arasında birden fazla yol çizilebilir demektir. Bu da ağaç olma şartını bozar.

Bir graf için birden fazla kapsayan ağaç tanımlanabilir. Bunların içinde toplam ağırlığı en küçük olana MST denir.

> **Hocanın vurgusu:** Bir graf için tek bir MST yoktur. Toplam ağırlığı aynı olan birden fazla farklı MST bulunabilir. Bu gayet normaldir, bunu unutmamamız gerekir.

2. Jenerik MST Algoritması ve Güvenli Kenar

2.1 Boş kümeden başlayıp kenar eklemek

MST'yi kenarlardan oluşan bir A kümesi olarak kuracağız. Başta elimizde hiçbir şey yoktur, bu yüzden A boş kümeyle başlar. Sonra graftan kenar seçip A'ya ekleyerek ağacı büyütürüz.

Burada sürekli korunması gereken bir koşul vardır: A kümesi her an, sonuçta oluşacak bir MST'nin alt kümesi olmalıdır. MST'de yer almayan bir kenar A'ya bir kez girerse artık MST elde edemeyiz.

Bu yüzden yalnız **güvenli kenarları** (safe edge) ekleriz. Güvenli kenar, eklendiğinde döngü oluşturmayan ve A'nın bir MST'nin alt kümesi olma özelliğini bozmayan kenardır. Hangi kenarı seçeceğimiz için akıllıca bir kenar seçim stratejisi gerekir. Her adımda, güvenli olma şartını sağlayan kenarlar arasından en ucuzunu seçerek ilerlersek, bütün düğümleri kapsayan ağaç oluştuğunda toplam ağırlığın en küçük olduğunu garanti edebiliriz. Buna açgözlü (greedy) çözüm denir.

Jenerik algoritma sözde kod olarak oldukça basittir:

```
GENERIC-MST(G, w)
  A = ∅
  A bir kapsayan ağaç olmadığı sürece
    A için güvenli bir (u, v) kenarı bul
    A = A ∪ {(u, v)}
  return A
```

Asıl soru şudur: Güvenli kenarı nasıl bulacağız?

2.2 Muhtarın sezgisi

Hoca slayttaki küçük bir köy örneğinde C ile F evleri arasındaki yolun maliyetinin 1 olduğunu gösterdi. Muhtar olsak bu iki evi bir şekilde bağlamamız gerekir. Şu an bunları bağlamanın en ucuz yolu 1'dir. Diyelim ki oluşan ağaçta C var ama F yok. F'yi bağlanmış evler arasına almamız lazım. Mevcut bağlantıların en ucuzunu seçeriz ki toplamdaki artış en az olsun. Yani her adımda, kümede olmayan düğümlere giden kenarlar içinden en düşük maliyetliyi seçip ekleyerek ağacı büyütürüz.

2.3 Kesim ve hafif kenar

Hoca güvenli kenar fikrini bir çizimle açtı. MST kurulurken bir t anında düğümler iki parçaya ayrılmış olsun: bir yanda S, öbür yanda $V - S$. Bu iki parçaya ayırmaya kesim (cut) denir. İki parçayı bir şekilde birbirine bağlamamız gerekir.

Diyelim ki aday olarak (x, y) kenarını seçmek istiyoruz. Bu kenar S ile $V - S$ arasında bir bağlantı kurar. Ama kesikli çizgiyle gösterilen bir (u, v) kenarı da olsun ve $w(u, v) < w(x, y)$ olsun. Mantık şunu gerektirir: Daha pahalı olan (x, y)'yi bağlamayız, iki parçayı (u, v) üzerinden bağlarız. Böylece toplamda daha ucuz bir çözüm elde ederiz ve ağaç yapısını da bozmayız.

Kısacası her adımda kesimin iki tarafını birbirine bağlayan kenarlar arasından en hafif olanı, yani hafif kenarı (light edge) seçmek, toplam ağırlığı en küçük tutar.

> **Ek bilgi (derste anlatılmadı):** Bu sezginin kitaptaki adı "kesim özelliği"dir (cut property). A'daki hiçbir kenarın kesmediği her kesim için, kesimi geçen en hafif kenar A için güvenlidir. Kruskal ve Prim, bu özelliği farklı kesimler seçerek uygular.

2.4 Bileşenlerden oluşan orman

Kenarları seçerek eklerken A kümesi, birbirine bağlı bileşenlerden oluşan bir orman (forest) oluşturur. İlk başta her düğüm ayrı bir bileşendir. Bir güvenli kenar seçince o kenarın iki ucundaki bileşenler birleşir ve tek bir bileşen (component) olur. Bu bileşen bir ağaç parçasıdır. Birden fazla ağaç bir arada durduğunda buna orman denir.

V düğümü bağlamak için $V - 1$ kenar gerekir. O halde $V - 1$ kez güvenli kenar eklersek elimizde tek bir bağlı bileşen (connected component) kalır. Hoca, jenerik algoritmayı tam olarak bu şekilde tasarlarsak Kruskal'ın algoritmasına ulaştığımızı söyledi.

3. Kruskal Algoritması

Kruskal'ın algoritması jenerik algoritmaya çok benzer. Hafif kenar seçme stratejisiyle ilerler:

- Her düğüm ayrı bir bileşen olarak başlar. Örneğin köyde 6 ev varsa 6 tekil bileşenle başlarız.
- Tekrar tekrar, iki farklı bileşeni birbirine bağlayan en hafif kenar bulunur ve eklenir.
- Bunun için kenarlar ağırlıklarına göre artan sırada taranır.
- Bir kenarın gerçekten iki ayrı bileşeni mi bağladığını, yoksa uçlarının zaten aynı bileşende mi olduğunu anlamak için ayrık kümeler (disjoint set) tutulur. Küme veri yapısıyla "bu düğümü önceden bağlamış mıydım?" kontrolü yapılır.

```
MST-KRUSKAL(G, w)
  A = ∅
  her v ∈ G.V için
    MAKE-SET(v)
  G.E kenarlarını w'ye göre artan sırada sırala
  her (u, v) ∈ G.E için (sıralı)
    eğer FIND-SET(u) ≠ FIND-SET(v)
      A = A ∪ {(u, v)}
      UNION(u, v)
  return A
```

3.1 Tahtadaki küçük örnek

Hoca algoritmayı önce elle çizdiği, sayısal ağırlık vermediği küçük bir örnekle anlattı. Köyde 5 ev ve aralarında 8 yol vardı. Başta 5 ayrı küme vardır.

- Kenarlar ağırlığa göre küçükten büyüğe sıralanır.
- En ucuz kenar (ağırlığı 1 olsun) seçilir. İki ucu farklı kümelerde olduğu için birleştirilir. Artık 4 küme vardır.
- İkinci en ucuz kenarın uçları da farklı kümelerdedir, bağlanır.
- Üçüncü en ucuz kenar (çapraz olan) eklenince solda 4 düğümlü bir küme, sağda tek düğümlü bir küme kalır.
- Sıradaki ucuz kenar (üstteki yatay kenar) kontrol edildiğinde iki ucunun zaten aynı kümede olduğu görülür. Bunlar daha ucuz kenarlarla zaten bağlanmıştır. Bu kenar pas geçilir.
- Sonraki uygun kenar eklenince 5 düğüm tek bir küme, tek bir bağlı bileşen olur.

Sonuçta oluşan ağaç en düşük toplam ağırlığa sahiptir, çünkü her adımda bileşenleri bağlayabilecek en hafif kenar eklenmiştir.

4. Kruskal ile Kitaptaki 9 Düğümlü Örnek

Ardından kitaptaki (CLRS) örnek işlendi. Köyde 9 ev vardır: $a, b, c, d, e, f, g, h, i$. Kenarlar ve ağırlıkları:

Kenar	Ağırlık	Kenar	Ağırlık
h-g	1	c-d	7
c-i	2	h-i	7

Kenar	Ağırlık	Kenar	Ağırlık
f-g	2	a-h	8
a-b	4	b-c	8
c-f	4	d-e	9
g-i	6	e-f	10
b-h	11	d-f	14

İlk iş, her düğümü ayrı bir küme yapmaktır: $\{a\}, \{b\}, \ldots, \{i\}$. Aynı anda kenarlar ağırlığa göre sıralanır. En hafif kenar 1 ağırlıklı h-g, en ağır kenar 14 ağırlıklı d-f'dir. Eşit ağırlıklı kenarlarda hoca alfabetik sırayı kullandı.

Adım adım işleyiş:

Adım	Kenar	Ağırlık	Karar ve bileşenler
1	h-g	1	Eklendi. {g, h}
2	c-i	2	Eklendi. {c, i}
3	f-g	2	Eklendi. {f, g, h}
4	a-b	4	Eklendi. {a, b}. Şu an 5 bileşen var: {a, b}, {c, i}, {f, g, h}, {d}, {e}
5	c-f	4	Eklendi. {c, i} ile {f, g, h} birleşti: {c, f, g, h, i}
6	g-i	6	Atlandı. g ile i aynı bileşende
7	c-d	7	Eklendi. {c, d, f, g, h, i}
8	h-i	7	Atlandı. Aynı bileşen
9	a-h	8	Eklendi. {a, b} ana bileşene bağlandı
10	b-c	8	Atlandı. Aynı bileşen
11	d-e	9	Eklendi. Tek bileşen kaldı

Hoca ikinci adımda c-i ile f-g'nin ikisinin de 2 olduğunu, hangisinin önce seçildiğinin fark etmediğini söyledi. Alfabetik olarak c önce geldiği için c-i önce alındı.

Altıncı adımda g-i kenarı üzerine hoca şunu söyledi: g ile i zaten aynı bileşendedir. Bu kenarı eklersek "fazladan yol yapmış oluruz". Adam i'den çıkıp c, f üzerinden g'ye zaten erişebiliyor. Bunları daha ucuz kenarlarla zaten bağlamıştık. Bu yüzden g-i güvenli bir kenar değildir.

Dokuzuncu adımda hoca beklenen bir soruyu kendisi sordu: "Hocam, a-h yerine b-c'yi seçseydik ne olurdu?" Hiçbir şey değişmezdi. Soldaki iki düğümlü bileşen {a, b}, sağdaki büyük bileşene bu sefer b-c üzerinden bağlanmış olurdu. Alfabetik olarak a-h önce geldiği için o seçildi.

4.1 Erken durdurma

d-e kenarı eklendiğinde tek bir bağlı bileşen kalır ve $V - 1 = 8$ kenar seçilmiş olur. Burada durulabilir, sonrasını kontrol etmeye gerek yoktur. Hoca, kenar ve düğüm sayısıyla uğraşıp ek bir kontrol yapmak istemezseniz bütün kenarları gezmeye devam edebileceğinizi söyledi. Bu durumda e-f (10), b-h (11) ve d-f (14) kenarlarının hepsi "aynı bileşende" denerek atlanır.

Seçilen kenarlar: h-g, c-i, f-g, a-b, c-f, c-d, a-h, d-e. Toplam ağırlık (ara adım eklendi):

$$1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37$$

4.2 Aynı graf, farklı MST

Hoca, eşit ağırlıklı kenarlarda seçim sırasını değiştirirse ne olacağını gösterdi. Bu sefer eşitlerde alfabetik sıranın tersiyle çalıştı ve ağacı mavi renkle çizdi:

- 1 ağırlıklı h-g yine seçildi.
- 2'lerin ikisi (f-g ve c-i) yine seçildi.
- 4'lerden önce c-f, sonra a-b seçildi. İkisi de eklendi.
- 6 ağırlıklı g-i ve 7 ağırlıklı h-i aynı bileşende kaldığı için seçilemedi. 7 ağırlıklı c-d seçildi.
- 8'lerden bu sefer b-c seçildi. Ardından a-h artık aynı bileşende kaldığı için seçilemedi.
- 9 ağırlıklı d-e seçildi.

Mavi ağacın kenarları: h-g, f-g, c-i, c-f, a-b, c-d, b-c, d-e. Toplamı yine 37'dir. Kırmızı ile çizilen ilk ağaçla mavi ağaç aynı değildir: birinde a-h, ötekinde b-c vardır. Ama toplam kenar ağırlıkları eşit olduğu için ikisi de MST'dir.

> **Hocanın vurgusu:** Muhtar olarak seçim sırasına bağlı olarak ya mavi olanı ya kırmızı olanı oluşturursunuz. İkisi de doğru cevaptır. Aynı graf için hep aynı MST'yi oluşturmak zorunda değiliz.
>
> **Ders sonrası not:** Birden fazla MST ancak eşit ağırlıklı kenarlar olduğunda ortaya çıkabilir. Bütün kenar ağırlıkları birbirinden farklıysa MST tektir.

5. Kruskal'ın Karmaşıklığı

Hoca algoritmanın parçalarının maliyetini sırayla saydı:

- Boş küme oluşturma: sabit zaman.
- Her düğüm için `MAKE-SET`: düğüm sayısı kadar iş, $O(V)$.
- Kenarları sıralama: E kenar en iyi ihtimalle $O(E \log E)$ sürede sıralanır.
- Kenar döngüsü: her kenar için "aynı kümede mi" kontrolü ve gerekirse birleştirme, kenar sayısı kadar, $O(E)$.

Hoca bu kısmı hızlı geçeceğini, parçaların nasıl birleştiğinin üzerinde çok durmayacağını söyledi. Özetle Kruskal'ın algoritması $O(E \log V)$ karmaşıklıktadır.

Kenarlar bize zaten sıralı verildiyse "çok şanslıyız". O zaman sıralama maliyeti ortadan kalkar ve karmaşıklık kenar sayısına bağlı, neredeyse doğrusal hale gelir. Sıralı değilse sıralama maliyeti üstümüze biner ve toplam $O(E \log V)$ olur.

> **Ek bilgi (derste anlatılmadı):** $E \leq V^2$ olduğu için $\log E \leq 2 \log V$'dir. Bu yüzden $O(E \log E)$ ile $O(E \log V)$ aynı mertebedir. Ayrık kümeler yol sıkıştırma (path compression) ve rütbeye göre birleştirme (union by rank) ile gerçekleştirilirse kenar döngüsü $O(E\,\alpha(V))$ sürer. Burada α çok yavaş büyüyen ters Ackermann fonksiyonudur. "Neredeyse doğrusal" ifadesi büyük olasılıkla buna karşılık gelir.

5.1 Toparlama: Kruskal nasıl çalışır?

Hoca Kruskal'ı şöyle özetledi:

- Bir kenar seçim stratejisi gerekir.
- V düğüm $V - 1$ kenarla bağlanmalıdır.
- Hiçbir şekilde döngü oluşturulmamalıdır. Bunun için güvenli kenarlar bulunur.
- Güvenli kenar tanımı: Bir kenarın iki ucu aynı kümedeyse onu tekrar bağlamak güvenli değildir, "boşuna masraftır". İki farklı bileşeni olabilecek en ucuz şekilde bağlayan kenar ise toplam maliyeti en az artıran, en güvenli çözümdür.

6. Prim Algoritması: Tek Ağacı Büyütmek

Prim'in algoritması MST algoritmalarının en çok bilinen ve en sık kullanılanlarından biridir. Bu algoritmalar tasarlayanların adıyla anılır: Kruskal MST, Prim MST.

6.1 Kruskal'dan farkı

Kruskal'daki temel mantık kenarlar üzerinden yürümek, farklı bileşenler oluşturmak ve bunları zaman içinde güvenli kenarlarla birleştirerek sonunda V düğümlü (yani $V-1$ kenarlı) tek bir bileşene ulaşmaktı.

Prim'de mantık bunun tersidir. Prim "herhangi bir t anında farklı ağaçlardan oluşan bir ormanla çalışmayayım" der. Her an tek bir bileşen, tek bir ağaç vardır. Seçilen A kümesi her zaman tek bir ağaç olma koşulunu sağlar. Her adımda öyle bir hafif kenar bulunur ki ağaca eklenince ağacı, kesimin öbür tarafındaki bir düğüme en hafif şekilde bağlar. Yani ağacın komşuları arasından en ucuz "asfalt yolu" seçe seçe, ekleye ekleye ağaç büyütülür.

Hoca bunu bir çizimle gösterdi. Ağaçtaki düğümlerin kümesi V_A olsun. Örnekte ağaçta 6 düğüm bağlanmış durumda. Kalan düğümler $V - V_A$ kümesindedir. Yedinci düğümü bağlamamız lazım. Ağaçtan bu düğüme giden iki kenar varsa ve sağdaki kenarın ağırlığı soldakinden azsa, sağdaki daha hafif kenardır ve onu tercih ederiz. Düğüm eklenince V_A 7 düğüme çıkar, $V - V_A$'da 6 düğüm kalır.

6.2 Hafif kenarı nasıl buluruz? Öncelikli kuyruk

Hoca burada sordu: Komşular arasındaki hafif kenarı nasıl bulabilirim?

Soru: Komşularımı zaten komşuluk listesinden bilmiyor muyum? Her seferinde komşuluk listesini tarar, en küçüğünü bulurum.

Cevap: Bulabilirsiniz, ama maliyetiniz yüksek olur. Her seferinde düğümün derecesi kadar işlem yapan bir doğrusal arama yapmış olursunuz. Daha iyisi, hafif kenarı en düşük maliyetle seçip çıkarmak için bir öncelikli kuyruk (priority queue) kullanmaktır.

Kuyruğun düzeni şöyledir:

- Kuyruktaki elemanlar, henüz ağaca eklenmemiş, yani $V - V_A$ kümesindeki düğümlerdir.
- Bir düğümün anahtar değeri (key), o düğümü ağaca bağlayan en hafif kenarın ağırlığıdır.
- Her adımda `EXTRACT-MIN` ile en hafif kenarla bağlanabilen düğüm çekilir.
- O anda ağaçla komşuluğu olmayan bir düğüm kuyrukta sonsuz (∞) anahtarla durur. "O düğüme komşuluğum olmadığı için gidemiyorum, gidiş masrafım sonsuzdur" deriz.
- Düğüm ağaca komşu hale gelince anahtarı azaltılır (`DECREASE-KEY`). Böylece düğüm min heap'te çıkması gereken yere kadar yukarı çıkar.

Hoca önce "increase key" dedi, hemen düzeltip "decrease key" olduğunu söyledi.

Algoritma rastgele seçilmiş bir r kök düğümünden başlar. Kökten komşulara bakılır, en hafif kenarla bağlanan komşu eklenir. Sonra oluşan ağacın bütün komşuları arasından tekrar en hafif olan eklenir. Toplam düğüm sayısı kadar adımda, erişilebilen bütün düğümlere ulaşıldığında işlem biter. Her adımda kuyruktaki en hafif kenar seçildiği için bu da açgözlü bir yaklaşımdır.

6.3 Sözde kod

Kruskal'dan farklı olarak Prim'in bir de başlangıç düğümüne (r) ihtiyacı vardır.

```
MST-PRIM(G, w, r)
  her u ∈ G.V için
    u.key = ∞
    u.π = NIL
  r.key = 0
  Q = G.V
  Q ≠ ∅ olduğu sürece
    u = EXTRACT-MIN(Q)
    her v ∈ G.Adj[u] için
      eğer v ∈ Q ve w(u, v) < v.key
        v.π = u
        v.key = w(u, v)      // DECREASE-KEY
```

- π (pi) ebeveyn (parent) bilgisidir: "Bu düğüme nereden geldim?" Başta `NIL`'dir.

- `r.key = 0` yapmak, kökü kuyruğun en tepesine, en öncelikli konuma taşır.
- Kuyruk boşalana kadar `EXTRACT-MIN` ile en hafif kenarla bağlanan düğüm çekilir. Komşularının anahtarı gerekirse $w(u, v)$'ye düşürülür ve ebeveynleri u yapılır.
- Bir düğüm eklenince yeni komşular gelir. Bunların içinden yine toplamdaki en hafif olan `EXTRACT-MIN` ile alınır.

  Ders sonrası not: Hoca ilklendirmeyi anlatırken "bütün kenarların anahtar değerlerini sonsuz atıyoruz", "her bir U kenarı için π" ve "kuyruğuma U ağırlıklı kenarı ekliyorum" dedi. Doğrusu düğümlerdir: her düğümün `key` değeri ∞, `π` değeri `NIL` yapılır ve kuyruğa düğümler konur.

7. Prim ile Aynı Köy Örneği

Hoca Kruskal'da kullanılan aynı köyü, aynı bağlantı grafını kullandı. Kök $r = a$ seçildi.

Adım adım işleyiş (ara anahtar değerleri anlatımdan kuruldu):

1. **a çekilir** (key 0). a'nın komşuları: b'ye 4, h'ye 8. Kuyrukta b: 4, h: 8.
2. **b çekilir** (key 4, kenar a-b). İkisi birlikte bir bileşen oluşturur, ağırlık 4. b ile yeni bir kenar gelir: c'ye 8. c'nin anahtarı ∞'dan 8'e düşer. Kuyrukta h: 8 (a üzerinden) ve c: 8 (b üzerinden) vardır.
3. **c çekilir** (key 8, kenar b-c). Hoca burada iki 8'in eşit olduğunu vurguladı: Gerçekleştirmeye bağlı olarak h'ye giden 8 de gelebilirdi. c'ye giden 8 daha sonra eklendiği ve `DECREASE-KEY` ile yukarı çıktığı için onu seçmiş olabilirsiniz. Hoca c'yi seçti. Ağaç a, b, c oldu. c sayesinde yeni komşular: d'ye 7, f'ye 4, i'ye 2.
4. **i çekilir** (key 2, kenar c-i). Kuyrukta 7 (d), 4 (f), 2 (i), 8 (h) vardı, en hafif 2'dir. Ağaç a, b, c, i oldu. i'den yeni kenarlar: g'ye 6, h'ye 7. h'ye gidiş 8'di, artık 7 olur.
5. **f çekilir** (key 4, kenar c-f). f'den yeni kenarlar: g'ye 2, d'ye 14, e'ye 10. g'nin anahtarı 6'dan 2'ye düşer. e'nin anahtarı ∞'dan 10 olur. d'nin anahtarı zaten 7 olduğu için 14 bir şey değiştirmez.
6. **g çekilir** (key 2, kenar f-g). Şu an 6 düğüm 5 kenarla bağlı: V düğüm $V - 1$ kenar. g'nin i ile komşuluğu zaten kendi bileşenidir. Ama g'den h'ye 1 vardır. h'nin anahtarı 7'den 1'e düşer ve min heap'te hemen köke çıkar.
7. **h çekilir** (key 1, kenar g-h). h'ye şimdiye kadar erişilmemişti, bağlandı.
8. **d çekilir** (key 7, kenar c-d). Kuyrukta 7 (d), 10 (e) kalmıştı. d'den e'ye 9 vardır. e'nin anahtarı 10'dan 9'a düşer, ebeveyni d olur.
9. **e çekilir** (key 9, kenar d-e). Kuyrukta yalnız e kalmıştı. Bağlanmamış son düğüm de o anki en hafif kenarla bağlandı.

Özet tablo:

Adım	Çekilen düğüm	Bağlandığı kenar	Ağırlık
1	a	(kök)	0
2	b	a-b	4
3	c	b-c	8
4	i	c-i	2
5	f	c-f	4
6	g	f-g	2
7	h	g-h	1
8	d	c-d	7
9	e	d-e	9

Toplam ağırlık (ara adım eklendi): $4 + 8 + 2 + 4 + 2 + 1 + 7 + 9 = 37$. Kruskal'daki ağaçlarla aynı toplamdır. Bu ağaç, Kruskal'ın mavi ağacıyla aynı kenarları içerir.

Hoca `DECREASE-KEY` etkisini e üzerinden tekrar gösterdi: e'ye önce 10 ile gidiliyordu. d bağlandığı anda e'ye 9 ile gidilebildi. Anahtar azaltıldı ve e'nin ebeveyninin d olduğu tutuldu. Her adımda sıralayıp en küçüğü almak ya da "bu kenara gitmiş miydim" kontrolleri yapmak da mümkündür, ama öncelikli kuyruk bunu düzenli yapar.

Ebeveyn dizisi sayesinde her düğüme nereden gelindiği izlenebilir:

Düğüm	a	b	c	i	f	g	h	d	e
π	NIL	a	b	c	c	f	g	c	d

> **Ders sonrası not:** Hoca ebeveynleri sayarken "B'nin parent'ı C" dedi, doğrusu `f.π = c`'dir. Ağacı tekrar anlatırken de "B'den G'ye, Y'den H'ye" dedi, doğrusu f'den g'ye ve g'den h'ye kenarlarıdır.

7.1 Erişilemeyen düğümler

Graf birbirinden bağımsız bileşenlerden oluşuyorsa Prim'in erişemeyeceği düğümler olabilir. Burada yönsüz graftan söz ediyoruz. Yönlü bir graf kullanılırsa, örneğin a'dan başlandığında bazı düğümlere hiç erişilemeyebilir. Erişilemeyen düğümler oluşan ağaçta yer almaz.

8. Prim'in Karmaşıklığı ve Kruskal ile Karşılaştırma

Öncelikli kuyruk derste görülen ikili heap (binary heap) ile gerçekleştirilirse:

- Kuyruğun ilklendirilmesi ve ilk `for` döngüsü: $O(V \log V)$.
- Kökün anahtarını ∞'dan 0'a azaltmak (`DECREASE-KEY`): $O(\log V)$. Bir eleman yapraktan köke en fazla $\log V$ karşılaştırmayla taşınır.
- Düğüm sayısı kadar `EXTRACT-MIN`: $O(V \log V)$.
- `DECREASE-KEY` sayısı kenar sayısından azdır, ama en kötü durumda üst sınır E alınır: $O(E \log V)$.

Toplam $O(V \log V + \log V + V \log V + E \log V)$'dir. Bunların en büyüğü $E \log V$ olduğu için Prim'in karmaşıklığı $O(E \log V)$ olarak verilir.

Daha etkili bir öncelikli kuyrukla, örneğin `DECREASE-KEY`'i sabit zamanda yapabilen Fibonacci heap ile, karmaşıklık $O(E + V \log V)$'ye iner. Hoca amortize analizin henüz işlenmediğini, bunların Algoritma Analizi dersinde görüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Fibonacci heap'te `DECREASE-KEY` amortize $O(1)$, `EXTRACT-MIN` amortize $O(\log V)$ sürer. V kez `EXTRACT-MIN` ve en fazla E kez `DECREASE-KEY` yapıldığından toplam $O(V \log V + E)$ olur. Yoğun graflarda (E, V^2 mertebesinde) bu fark belirginleşir.
>
> **Hocanın vurgusu:** Prim ya da Kruskal kullandığınızda karmaşıklık açısından "aman aman bir kazancınız yok". Ama çalışma şekilleri tamamen farklıdır.

	Kruskal	Prim
Başlangıç	V ayrı bileşen (küme)	Tek bir kök düğüm
Her adım	En hafif güvenli kenarla iki bileşeni birleştirir	Ağacı en hafif kenarla bir düğüm büyütür
Ara durum	Orman	Her an tek ağaç
Yardımcı yapı	Ayrık kümeler, sıralı kenar listesi	Öncelikli kuyruk (min heap)
Karmaşıklık	$O(E \log V)$	$O(E \log V)$, Fibonacci heap ile $O(E + V \log V)$

MST için başka algoritmalar da tartışılabilir, ama temelde aktarılabilecek iki yaklaşım bunlardır. MST graflarda önemli bir konudur. Pek çok algoritmanın temelini oluşturur. Bir graftan MST çıkarmak birçok problemde ilk adım, bir ön işleme (preprocessing) aşaması gibi çalışır. Bu yüzden MST'yi etkili biçimde elde etmek önemlidir.

9. Tek Kaynaklı En Kısa Yol Problemi

Hoca, Prim'e oldukça benzeyen başka bir probleme geçti: en kısa yol problemi. Bir kaynaktan başlayarak erişebildiğimiz diğer düğümlere en kısa yoldan nasıl gideriz? Buna tek kaynaklı en kısa yol (single source shortest path) denir.

9.1 Girdi ve amaç

- Bir $G = (V, E)$ grafı gerekir.
- Kenar ağırlıklı olmalıdır. Ağırlıksız grafta yalnız kaç kenar geçildiği sayılabilir, ama en kısa yol problemlerinde kenar ağırlıkları tanımlıdır.
- Grafın yönlü olup olmaması isteğe bağlıdır. Yönsüz graflarda da en kısa yol taraması yapılabilir.
- Bir s kaynak (source) düğümü verilir.
- Amaç: s'den yolu olan bütün düğümlere, toplam ağırlığı en küçük olan rotayı bulmak.

Ağırlığın anlamı probleme göre değişir. Kavşaklar düğüm, yollar kenar olarak verilirse yolun kilometresi ağırlık olabilir. Yolun kaç dakikada gidildiği verilirse toplam süre ağırlık olur. Bu durumda "en hızlı yol" ya da "en kısa yol" sorulmuş olur. Hoca, problemin adı "shortest path" olduğu için genelde kilometre üzerinden düşünmeyi daha mantıklı bulduğunu söyledi.

9.2 Gezgin satıcı problemiyle karıştırmamak

> **Hocanın vurgusu:** Bu problem, kaynaktan başlayıp bütün düğümleri en kısa şekilde nasıl gezerim problemi değildir. O, gezgin satıcı problemidir (traveling salesman problem).

Gezgin satıcı probleminde bir şehirden başlanır, bütün şehirler gezilir ve genelde başlanan şehre geri dönülür. Toplam yolu en küçük yapan gezme sırası aranır. Burada ise daha basit bir şey yapıyoruz: A'ya en kısa nasıl giderim, B'ye en kısa nasıl giderim, C'ye en kısa nasıl giderim, diye her düğüm için ayrı ayrı soruyoruz. Bu yolların oluşturduğu ağa tek kaynaklı en kısa yollar denir.

Tek bir kaynaktan değil de bütün düğümlerden bütün düğümlere en kısa yollar istenirse problemin adı tüm çiftler arası en kısa yol (all pairs shortest path) olur. Hoca bundan bu derste bahsetmeyeceğini, orada örneğin Floyd algoritmasının kullanılabileceğini söyledi.

9.3 Kullanım alanları

- **Ulaşım planlama:** İşten eve ya da Yıldız kampüsünden Davutpaşa kampüsüne en kısa mesafeden gitmek.
- **Ağlar:** İnternette bir kaynak bilgisayardan hedef bilgisayara veri paketlerinin hangi switch ve router'lar üzerinden en hızlı şekilde gideceği.
- **Sosyal ağ analizi:** Bir kişinin bir diğerine ne kadar yakın olduğu.
- Ses tanıma, metin ve belge işleme, robotik, derleyiciler.
- Neredeyse bütün zamanlama (scheduling) problemleri ve havayolu ekip planlaması gibi kaynak atama problemleri.

Ses örneği için hoca şunu anlattı: Sesin özellikleri ve frekansları kullanılarak bir müzik eseri ya da ses sinyali modellenir. Ses kaynakları ve aralarındaki uzaklıklar, ya da frekanslar ve birbirlerine uzaklıkları düğüm ve kenar olarak bir graf oluşturur. Sonra "bu ses öbürüne ne kadar benziyor", "hangi enstrüman hangisine daha yakın" gibi sorular en kısa yolla incelenebilir. Hoca ses işleme üzerine çalışmadığı için ayrıntısını bilmediğini belirtti.

9.4 Kaba kuvvet neden olmaz?

Hoca harita uygulamasındaki bir rota önerisini gösterdi: Beşiktaş'taki kampüsten Davutpaşa kampüsünün kapısına E5'ten 23 dakika, sahilden ve tünellerden geçip tekrar E5'e bağlanan gri rota 25 dakika, otobandan ve otogar üzerinden giden rota 26 dakika. Kavşaklar düğüm, aralarındaki yollar kenar olarak modellenir.

Kaba kuvvet (brute force) yaklaşımı şöyle olurdu: Kaynak ile hedef (sink) arasındaki olası bütün rotaları çıkarırım. Saçma da olsa bütün İstanbul turu yapan rotalar dahil. Her rotanın geçtiği kenarların kilometresini ya da dakikasını toplarım, sıralarım ve en küçüğünü seçerim. Bunun sonu yoktur. Harita uygulamaları bu yöntemi kullansaydı hiçbirimiz A'dan B'ye rota oluşturamazdık. Soru şudur: Graflarla bunu daha etkili nasıl çözeriz?

10. Dijkstra Algoritması

10.1 Tarihçe

Bu problemi bilgisayar biliminin önemli isimlerinden Dijkstra çözmüştür. Dijkstra Hollandalı bir bilgisayar bilimci ve matematikçidir, 1930 doğumludur, 2002'de vefat etmiştir. Algoritmayı 1956'da tasarlamıştır. Hocanın çok sevdiği sözü şudur:

> "This was the first graph problem I ever posed myself and solved."

Yani kendi kendine "verilen bir grafta şu noktadan başlayarak en kısa yolları nasıl bulurum?" diye sormuş ve çözmüştür. İlginç olan, çözümünü hemen yayımlamamasıdır. O zamanlar bu tür işler ilgi çekici görülmüyordu, algoritma tasarımı bilimsel bir başlık olarak pek ele alınmıyordu. 1956'da tasarladığı algoritmayı 1959'da yayımlamıştır.

10.2 Koşullar

- Dijkstra hem yönlü hem yönsüz graflarda çalışır.
- Önemli kısıt: Bütün kenar ağırlıkları için $w(u, v) \geq 0$ olmalıdır.

Soru: Negatif ağırlık nasıl oluyor?

Cevap: Elektrik devrelerinde ya da bazı kimya ağlarında (elektron akışları gibi) negatif ağırlık olabilir. Grafta negatif ağırlık varsa Dijkstra çalışmaz. Neyse ki orada Dijkstra'ya biraz benzeyen Bellman-Ford algoritması vardır. Negatif ağırlıklı graflarda en kısa yol onunla bulunur. Bu ders kapsamında işlenmeyecek.

> **Ders sonrası not:** Hoca önce kenar ağırlıklarının "pozitif" olması gerektiğini söyledi, sonra $w(u, v) \geq 0$ yazdı. Doğru koşul "negatif olmayan" ağırlıktır. 0 ağırlıklı kenar sorun çıkarmaz.
>
> **Ek bilgi (derste anlatılmadı):** Dijkstra, kuyruktan çekilen düğümün uzaklığının artık kesinleştiğini varsayar. Negatif bir kenar varsa, sonradan çekilen bir düğüm üzerinden daha önce kesinleşmiş bir düğüme daha kısa bir yol bulunabilir. Bu varsayım bozulduğu için sonuç yanlış çıkabilir.

10.3 Prim ile benzerlik ve fark

Prim'de olduğu gibi Dijkstra'da da iki düğüm kümesi vardır:

- S: en kısa yolları hesaplanmış (kesinleşmiş) düğümler.
- Q: $V - S$'deki, yani kalan düğümlerden oluşan öncelikli kuyruk.

Fark şudur: Prim'de anahtar değer tek bir kenarın ağırlığıydı. Dijkstra'da anahtar değer, kaynaktan o düğüme kadar olan yolun toplam ağırlığıdır.

10.4 Sözde kod ve gevşetme

```
DIJKSTRA(G, w, s)
  INITIALIZE-SINGLE-SOURCE(G, s)
  S = ∅
  Q = G.V
  Q ≠ ∅ olduğu sürece
    u = EXTRACT-MIN(Q)
    S = S ∪ {u}
    her v ∈ G.Adj[u] için
      RELAX(u, v, w)

INITIALIZE-SINGLE-SOURCE(G, s)
  her v ∈ G.V için
    v.d = ∞
    v.π = NIL
  s.d = 0
```

```
RELAX(u, v, w)
  eğer v.d > u.d + w(u, v)
    v.d = u.d + w(u, v)
    v.π = u
```

İlklendirmede bütün düğümlerin uzaklığı (distance) sonsuz, ebeveyni `NIL` yapılır. Kaynağın uzaklığı 0'dır: "Başladığım s düğümüne erişmek için harcadığım yol toplam sıfırdır."

Gevşetme (relax) işlemi şunu sorar: Bu kenarı seçip yoluma eklemem, önceki yolumdan toplamda daha ucuz bir maliyet getiriyor mu? Getiriyorsa o düğüme gidiş yolumu bu yeni yolla değiştiririm. Hocaya göre Prim'den temel fark aslında budur.

10.5 Karmaşıklık

- Öncelikli kuyruk dizi ile yapılırsa: $O(V^2)$.
- Min heap ile yapılırsa: $O(E \log V)$, Prim'deki gibi.
- Fibonacci heap ile: $O(E + V \log V)$.

Sebebi, Dijkstra'nın Prim'den tek farkının anahtar değer olarak o anki kenarı değil, o ana kadarki kenarların toplam ağırlığını kullanmasıdır. Kuyruk işlemlerinin sayısı aynı kalır.

11. Dijkstra Örnek 1: Elle Çizilmiş Yönlü Graf

Hoca önce elle çizilmiş basit bir yönlü graf gösterdi. Kaynak A'dır. A'dan B, C, D, E düğümlerinin hepsine farklı farklı yollarla erişilebilir. Örneğin E'ye A-C-E ya da A-B-E üzerinden gidilebilir. Kenar ağırlıkları kilometre olarak düşünüldü:

- A→B 4, A→C 2
- C→B 1, C→D 4, C→E 5
- B→C 3, B→D 2, B→E 3
- E→D 1

İşleyiş:

1. **İlklendirme:** Öncelikli kuyruk oluşturulur. A'nın uzaklığı 0'dır, "çünkü zaten orada duruyorum". B, C, D, E'nin uzaklığı ∞'dur.
2. **A çekilir (0).** A→B ile B'ye $0 + 4 = 4$, ebeveyni A. A→C ile C'ye $0 + 2 = 2$, ebeveyni A.
3. **C çekilir (2).** Kuyruktaki en küçük toplam yol 2'dir.
   - C→B: $2 + 1 = 3 < 4$. Eskiden B'ye doğrudan 4 ile gidiliyordu. A-C-B rotası 3 km'dir. B güncellenir: 3, ebeveyni C.
   - C→D: $2 + 4 = 6$. D: 6, ebeveyni C.
   - C→E: $2 + 5 = 7$. E: 7, ebeveyni C.
4. **B çekilir (3).** Kuyrukta 3, 6, 7 vardı.
   - B→C: $3 + 3 = 6 > 2$. C'ye zaten 2 ile gidiliyor. Gevşetme bunu kontrol eder, C değişmez.
   - B→D: $3 + 2 = 5 < 6$. D: 5, ebeveyni B.
   - B→E: $3 + 3 = 6 < 7$. E: 6, ebeveyni B.
5. **D çekilir (5).** D'den hiçbir yere gidilemez. Yapılacak bir şey yoktur.
6. **E çekilir (6).** E→D: $6 + 1 = 7 > 5$. Bu yol tercih edilmez, eski yol korunur.

Adım	Çekilen	d[B]	d[C]	d[D]	d[E]
Başlangıç	—	∞	∞	∞	∞
1	A (0)	4 (A)	2 (A)	∞	∞
2	C (2)	3 (C)	2	6 (C)	7 (C)
3	B (3)	3	2	5 (B)	6 (B)
4	D (5)	3	2	5	6
5	E (6)	3	2	5	6

Parantez içi ebeveyni gösterir.

11.1 Yolu geri okumak

Sonuçta C'ye 2 km, B'ye 3 km, D'ye 5 km, E'ye 6 km ile gidilir. Yollar ebeveyn bilgisinden geriye doğru okunur:

- B'nin ebeveyni C, C'nin ebeveyni A: A → C → B.
- D'nin ebeveyni B: A → C → B → D.
- E'nin ebeveyni B: A → C → B → E. E'ye gitmek için B'den gelinmeli, B'ye C'den, C'ye A'dan. A kaynak olduğu için orada durulur.

Hoca, B için ilk bulunan 4'ün mantıklı göründüğünü, ama sonra daha ucuz 3'lük bir yol bulunduğunu vurguladı.

12. Dijkstra Örnek 2: Kitaptaki Yönlü Graf

İkinci örnek kitaptandır. Kaynak s (start ya da source). Kenarlar:

- s→t 10, s→y 5
- t→y 2, t→x 1
- y→t 3, y→x 9, y→z 2
- x→z 4
- z→x 6, z→s 7

Hoca bu örneği tablo yerine graf üzerinde çizerek işledi. İşlenen düğümleri siyaha boyadı.

1. **Başlangıç:** $d[s] = 0$, diğer bütün düğümler ∞.
2. **s çekilir (0).** t'ye $0 + 10 = 10$, y'ye $0 + 5 = 5$.
3. **y çekilir (5).** 5 ile 10 arasından 5 gelir.

- y→t: $5 + 3 = 8 < 10$. t: 8, ebeveyni y. Yani s-y-t yolu 8, s-t yolu 10'dur.
- y→x: $5 + 9 = 14$. x: 14.
- y→z: $5 + 2 = 7$. z: 7.

4. **z çekilir (7).** Kuyrukta 7, 8, 14 vardı.

- z→s: $7 + 7 = 14$. s'nin uzaklığı zaten 0'dır. Hoca "bu saçma" diyerek bunu kullanmadı.
- z→x: $7 + 6 = 13 < 14$. x: 13, ebeveyni y'den z'ye değişti.

5. **t çekilir (8).** Kuyrukta 8 ve 13 vardı.

- t→y: $8 + 2 = 10 > 5$. y'ye zaten 5 ile gidilmişti, kullanılmaz.
- t→x: $8 + 1 = 9 < 13$. x: 9, ebeveyni t.

6. **x çekilir (9).** x→z: $9 + 4 = 13 > 7$. z'ye 7 zaten daha ucuz, bu yol atılır.

Sonuç:

Düğüm	s	y	z	t	x
Uzaklık	0	5	7	8	9
Ebeveyn	NIL	s	y	y	t
Yol	—	s→y	s→y→z	s→y→t	s→y→t→x

Hoca, graf üzerinde çizmenin daha kompakt ve anlaşılır olduğunu, adım adım işletmek için ise ilk örnekteki tablonun daha uygun olduğunu söyledi.

13. Dijkstra Örnek 3: Yönsüz Graf ve A'dan Z'ye Yol

Son örnekte hoca Dijkstra'nın yönsüz graflarda da çalıştığını gösterdi. Soru "A'dan erişilebilen bütün düğümlere en kısa yollar" olarak da, "A'dan Z'ye en kısa yol nedir?" olarak da sorulabilir. Yıldız kampüsünden Davutpaşa kampüsüne en kısa yol gibi düşünülebilir. Kenarlar:

- A-B 4, A-C 2
- B-C 1, B-D 5
- C-D 8, C-E 10
- D-E 2, D-Z 6

- E-Z 5

İşleyiş (slaytta erişilemeyen düğümler mor renkteydi):

1. **A çekilir (0).** B: $0 + 4 = 4$, önceli (previous) A. C: $0 + 2 = 2$, önceli A.
2. **C çekilir (2).**
   - B: $2 + 1 = 3 < 4$. B: 3, önceli C.
   - D: $2 + 8 = 10$. D: 10, önceli C.
   - E: $2 + 10 = 12$. E: 12, önceli C.
3. **B çekilir (3).** Kuyrukta 3, 10, 12 vardı. A ve C önceden işlenmiş (visited) düğümlerdir.
   - D: $3 + 5 = 8 < 10$. D: 8, önceli B.
4. **D çekilir (8).** Kuyrukta 8 ve 12 vardı.
   - E: $8 + 2 = 10 < 12$. E: 10, önceli D.
   - Z: $8 + 6 = 14$. Z: 14, önceli D.
5. **E çekilir (10).** Z: $10 + 5 = 15 > 14$. Z değişmez, önceli D kalır. Yani E üzerinden Z'ye gitmek gereksizdir.
6. **Z çekilir (14).** Z'den gidilebilecek başka yer yoktur. Durulur.

Çekilen	Güncellemeler
A (0)	B = 4 (A), C = 2 (A)
C (2)	B = 3 (C), D = 10 (C), E = 12 (C)
B (3)	D = 8 (B)
D (8)	E = 10 (D), Z = 14 (D)
E (10)	Z için $10 + 5 = 15 > 14$, değişmez
Z (14)	Bitti

> **Ders sonrası not:** Hoca B'den D'ye geçerken "B'ye gidiş maliyetim neydi? 8'di" dedi. B'nin uzaklığı 3'tür. 8, D'nin yeni uzaklığıdır: $3 + 5 = 8$.

A'dan Z'ye en kısa yol Z'den başlayıp önceller dizisinde geri gidilerek bulunur: Z'nin önceli D, D'nin önceli B, B'nin önceli C, C'nin önceli A. A kaynaktır, orada durulur. Yol A → C → B → D → Z, uzunluğu 14'tür.

Örnek Sorular

Bu derste ekrandan alınmış soru yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu ve cevapladığı sorulardır.

1. (derste sözlü, 00:13:29) V düğümlü bir graf için MST'de kaç kenar bulunur?

> **Çözüm:** V düğümü bağlamak gerekir. Ağaç olduğu için $V - 1$ kenar yeter. $V - 1$ kez güvenli kenar eklenince elde tek bir bağlı bileşen kalır.

2. (derste sözlü, 00:27:11) Kitaptaki 9 düğümlü Kruskal örneğinde, 8 ağırlıklı kenarlardan a-h yerine b-c seçilseydi ne olurdu?

> **Çözüm:** Hiçbir şey değişmezdi. {a, b} bileşeni büyük bileşene a-h yerine b-c üzerinden bağlanırdı. Diğer 8'lik kenar artık aynı bileşende kalacağı için seçilmezdi. Ortaya farklı ama yine 37 ağırlıklı bir MST çıkar.

3. (derste sözlü, 00:39:11) Prim'de ağacın komşuları arasındaki hafif kenarı nasıl buluruz? Her seferinde komşuluk listesini taramak yeterli mi?

> **Çözüm:** Taramak doğru sonucu verir, ama her seferinde doğrusal arama olduğu için pahalıdır. Henüz ağaçta olmayan düğümler, anahtarı "ağaca bağlayan en hafif kenarın ağırlığı" olan bir öncelikli kuyrukta (min heap) tutulur. Hafif kenar `EXTRACT-MIN` ile alınır.

4. (derste sözlü, 00:40:40) Prim'de o anki ağaçla komşuluğu olmayan bir düğüm öncelikli kuyrukta nasıl yer alır?

Çözüm: Anahtarı ∞ olarak yer alır. Düğüm ağaca komşu olunca anahtarı `DECREASE-KEY` ile kenar ağırlığına düşürülür.

5. (derste sözlü, 01:33:45) Negatif ağırlıklı kenarlar varsa Dijkstra kullanılabilir mi?

Çözüm: Hayır. Dijkstra $w(u, v) \geq 0$ şartı ister. Negatif ağırlıklı graflarda en kısa yol Bellman-Ford algoritmasıyla bulunur.

6. (derste sözlü, 02:06:36) Örnek 3'teki yönsüz grafta A'dan Z'ye en kısa yol nedir?

Çözüm: Bölüm 13'teki işleyişe göre uzaklıklar A = 0, C = 2, B = 3, D = 8, E = 10, Z = 14'tür. Önceller dizisinden geri okunan yol A → C → B → D → Z, uzunluğu 14'tür.

Dersin Toparlaması

Bu derste graflarda iki temel problem işlendi. İlki en küçük kapsayan ağaçtır. Jenerik algoritma boş bir kümeden başlar ve her adımda yalnız güvenli kenar ekler. Güvenli kenar, bir kesimin iki tarafını en ucuz şekilde bağlayan hafif kenardır. Kruskal bu fikri V ayrı bileşenle başlayıp kenarları artan sırada tarayarak ve ayrık kümelerle döngü kontrolü yaparak uygular. Prim ise her an tek bir ağaç tutar ve bu ağacı öncelikli kuyruktan çekilen en hafif kenarla büyütür. İkisi de $O(E \log V)$'dir ve aynı graf için toplam ağırlığı aynı, ama kenarları farklı olabilen MST'ler üretebilir. Kitaptaki köy örneğinde bütün çözümlerin toplamı 37 çıktı.

İkinci problem tek kaynaklı en kısa yoldur. Bu problem gezgin satıcı problemi değildir: her düğüme kaynaktan ayrı ayrı en kısa yol aranır. Dijkstra algoritması Prim'in yapısını kullanır, ama anahtar değer olarak tek kenarın ağırlığı yerine kaynaktan gelen toplam yolu kullanır. Uzaklıklar gevşetme ile güncellenir. Kenar ağırlıklarının negatif olmaması gerekir. Negatif ağırlıkta Bellman-Ford kullanılır. Yollar, ebeveyn dizisinde hedeften kaynağa geri gidilerek bulunur. Tüm çiftler arası en kısa yol (örneğin Floyd algoritması) ve amortize analizle Fibonacci heap ayrıntıları ileri derslere bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.